PROGRAMACIÓN DECLARATIVA CLASE
PRÁCTICA 4

Tema: *Corte y Negación*

1. Dada la definición del predicado disjuntas/2, que verifica si dos listas no tienen ningún elemento en común:

```
(D1) disjuntas(L1, L2) :- not ((member(X, L1), member(X, L2))).
```

   Construya los árboles de derivación de los siguientes objetivos:

```
a) :- disjuntas([1, 2],[3]).
b) :- not(disjuntas([2,3],[2])).
```

2. Dada la definición del predicado `union/3,` que halla la unión de dos listas:

```
(U1) union([],X,X):-!.
(U2) union([X|Y],Z,[X|W]):-not(member(X,Z)),union(Y,Z,W),!.
(U3) union([X|Y],Z,W):-member(X,Z),union(Y,Z,W).
```

   Construya los árboles de derivación de los siguientes objetivos:

```
a) :-not(union([4],[2,3],[X|Y])).
b) :-not(union([X],[2,3],[1|Y])).
```

3. Defina en Prolog el predicado `interseccion(L1,L2,R)` donde R es la lista que contiene los elementos comunes de las listas L1 y L2.
   a) Analice cuantas respuestas da su predicado por RC (retroceso cronológico).
   b) Si las tres variables están instanciadas, ¿Su predicado triunfa en los casos en que debe hacerlo?

4. Dada la siguiente definición de `merge/3`:

```
(M1) merge(X,[],X).
(M2) merge([],X,X).
(M3) merge([X|X1],[Y|Y1],[X|Z]):- X < Y,merge(X1,[Y|Y1],Z).
(M4) merge([X|X1],[Y|Y1],[X,Y|Z]):- X = Y,merge(X1,Y1,Z).
(M5) merge([X|X1],[Y|Y1],[Y|Z]):- X > Y,merge([X|X1],Y1,Z).
```

   a) Analice la deficiencia del intérprete con dicha definición (que no utiliza el corte) en el siguiente objetivo:

```
:- merge([1,3],[2],X),[2|Y]=X.
```

   b) Muestre cómo quedaría el árbol de derivación-SLD utilizando el corte.

5. Reescriba el siguiente programa dándole un uso adecuado al corte. Ponga un ejemplo de por qué está mal definido.

```
max(X,Y,Y):- X=<Y,!.
max(X,Y,X).
```

6. Defina un predicado para adicionar un elemento a una lista en caso de que el elemento no esté en la lista. Realice dos definiciones, una utilizando el corte y otra sin él. Analice la eficiencia de ambas definiciones.

7. Defina en Prolog los predicados del ejercicio 4 de la CP anterior que describen operaciones sobre listas, haciendo uso del corte.

   a) aplanar(L1,L2) donde L1 puede contener como elementos a otras listas y L2 es la lista L1 en forma aplanada. Ejemplo:
   :-aplanar([[5],1,[[2,3,[]],7],4],X).
   X = [5,1,2,3,7,4].

   b) divide(L,I,P) que divide la lista L colocando los elementos de las posiciones impares en I y los de las pares en P. Suponga que las listas comienzan en la posición 1.

   c) reemplazar(X,Y,L,R) que reemplaza todas las ocurrencias de X que estén en la lista L por Y y da en R la lista resultante.

   d) comprimir(L1,L2) que elimina las repeticiones consecutivas de un elemento. Ejemplo:
   :-comprimir([a,a,b,b,c,c,a,a,d],L2).
   L2 = [a,b,c,a,d].

   e) empaquetar(L1,L2) que agrupa en listas las ocurrencias consecutivas de un elemento. Ejemplo:
   :-empaquetar([a,a,b,b,c,c,a,a,d],L2).
   L2 = [[a,a],[b,b],[c,c],[a,a],[d]].

   f) prefijo(L1,L2,P) que da en la lista P el prefijo común de las listas L1 y L2.

   g) sufijo(L1,L2,S) que da en la lista S el sufijo común de las listas L1 y L2.

   h) eliminatodos(E,L1,L2) que da en la lista L2 la lista L1 sin todas las ocurrencias del elemento E.